



704. Binary Search

Description:

- ✓ Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`.
- ✓ If `target` exists, then return its index. Otherwise, return `-1`.
- ✓ You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`

Output: `4`

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`

Output: `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are unique.
- `nums` is sorted in ascending order.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 3

Amazon 2

Adobe 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google **7**

Uber **4**

Facebook **2**

Yahoo **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft **8**

Bloomberg **5**

SAP **3**

Yandex **3**

tcs **3**

Goldman Sachs **2**

Infosys **2**

704. Binary Search

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int search(int[] arr, int element) {
        int n = arr.length;
        int low = 0;    int high = n - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (element == arr[mid] ) {
                return mid;
            }
            else if (element > arr[mid]) {
                low = mid + 1;
            }
            else if (element < arr[mid]) {
                high = mid - 1;
            }
        }
    }
}
```



```
        return -1;  
    }  
}
```

34. Find First and Last Position of Element in Sorted Array

Description:

- ✓ Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.
- ✓ If target is not found in the array, return [-1, -1].
- ✓ You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]

Example 2:

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- nums is a non-decreasing array.
- $-109 \leq \text{target} \leq 109$

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Adobe 4 Apple 2 Facebook 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 14 Bloomberg 5 Intuit 4 Google 3 Uber 2

Samsung 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft 17 LinkedIn 16 Qualtrics 4 ByteDance 4 instacart 3

Yahoo 2 Nutanix 2 Oracle 2 Flipkart 2 Splunk 2 TikTok 2

SAP 2

34. Find First and Last Position of Element in Sorted Array

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    private static int firstOccurrence(int arr[], int element) {

        int n = arr.length;
        int low = 0;
        int high = n - 1;
    }
}

```



```
        while (low <= high) {
            int mid = (low + high) / 2;
            if (element > arr[mid]) {
                low = mid + 1;
            } else if (element < arr[mid]) {
                high = mid - 1;
            } else {
                if (mid == 0 || arr[mid - 1] != arr[mid])
                    return mid;
                else
                    high = mid - 1;
            }
        }
        return -1;
    }
```

```
private static int lastOccurrence(int arr[], int element) {

    int n = arr.length;
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (element > arr[mid]) {
            low = mid + 1;
        } else if (element < arr[mid]) {
            high = mid - 1;
        } else {
            if (mid == n - 1 || arr[mid] != arr[mid + 1])
                return mid;
            else
                low = mid + 1;
        }
    }
    return -1;
}
```



```
public int[] searchRange(int[] arr, int element) {  
    int first = firstOccurrence(arr, element);  
  
    if(first==-1) {  
        return new int[]{-1, -1};  
    }  
  
    int last = lastOccurrence(arr,element);  
    return new int[]{first, last};  
}
```

69. Sqrt(num)

Description:

- ✓ Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.
- ✓ You must not use any built-in exponent function or operator.

Example 1:

Input: x = 4 Output: 2

Example 2:

Input: x = 8 Output: 2

Constraints:

a) $0 \leq x \leq 2^{31} - 1$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **6**

Apple **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg **5**

Microsoft **3**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

LinkedIn **15**

Google **7**

Adobe **7**

Uber **5**

Facebook **3**

TikTok **2**



69. Sqrt(num)

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public int mySqrt(int num) {

        if (num < 2)
            return num;

        int low = 2;
        int high = num/2;
        long result=0;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            result = (long)mid * mid;

            if (result == num) {
                return mid;
            } else if (result > num) {
                high = mid - 1;
            } else if (result < num) {
                low = mid + 1;
            }
        }

        return high;
    }
}
```



33. Search in Rotated Sorted Array

Description:

- ✓ There is an integer array `nums` sorted in ascending order (with distinct values).
- ✓ Prior to being passed to your function, `nums` is possibly rotated at an unknown pivot index `k` ($1 \leq k < \text{nums.length}$)
- ✓ For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index 3 and become `[4,5,6,7,0,1,2]`.
- ✓ Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.
- ✓ You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0` Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3` Output: -1

Constraints:

- a) $1 \leq \text{nums.length} \leq 5000$
- b) $-104 \leq \text{nums}[i] \leq 104$
- c) All values of `nums` are unique.
- d) `nums` is an ascending array that is possibly rotated.
- e) $-104 \leq \text{target} \leq 104$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple **5**

Amazon **3**

Google **3**

Uber **3**

Yahoo **2**

Oracle **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg **11**

Adobe **6**

Microsoft **5**

Media.net **5**

TikTok **3**

IXL **2**

Facebook **2**

PayPal **2**

Visa **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

LinkedIn **40**

ByteDance **10**

VMware **6**

Walmart Labs **6**

Goldman Sachs **6**

ServiceNow **6**

Expedia **5**

Flipkart **3**

Salesforce **3**

Morgan Stanley **3**

Splunk **3**

Nvidia **2**

Airbnb **2**

Intuit **2**

Paytm **2**

Qualtrics **2**

MakeMyTrip **2**

Airtel **2**

33. Search in Rotated Sorted Array

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```

```
class Solution {
    public int search(int[] arr, int element) {
```



```
int n = arr.length;
int low = 0;
int high = n - 1;

while (low <= high) {
    int mid = (low + high) / 2;

    if (element == arr[mid]) {
        return mid;
    }

    else if (arr[mid] >= arr[low]) {
        if (element >= arr[low] && element < arr[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    else {
        if (element <= arr[high] && element > arr[mid]) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
}

return -1;
}
```



162. Find Peak Element

Description:

- ✓ A peak element is an element that is strictly greater than its neighbors.
- ✓ Given a 0-indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.
- ✓ You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: nums = [1,2,3,1] Output: 2

Example 2:

Input: nums = [1,2,1,3,5,6,4] Output: 5

Constraints:

- a) $1 \leq \text{nums.length} \leq 1000$
- b) $-231 \leq \text{nums}[i] \leq 231 - 1$
- c) $\text{nums}[i] \neq \text{nums}[i + 1]$ for all valid i .

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 5

Apple 2

Facebook 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google **3**

Microsoft **3**

Uber **3**

Adobe **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Roblox **12**

HRT **7**

Snapchat **4**

Bloomberg **3**

Walmart Labs **3**

Yahoo **2**

IXL **2**

VMware **2**

PayPal **2**

TikTok **2**

162. Find Peak Element

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int findPeakElement(int[] arr) {
        int low = 0;
        int high = arr.length - 1;
        while (low < high) {
            int mid = (low+high) / 2;
            if (arr[mid] > arr[mid + 1])
                high = mid;
            else
                low = mid + 1;
        }
        return low;
    }
}
```